SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME:Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

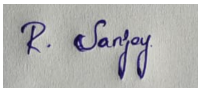
Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

I Sanjay Raghavendra/ 192411041 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

ANSWERS :

QUESTION 1: Error Analysis in Equivalence Class Partitioning and Boundary Value Analysis

Introduction

Equivalence Class Partitioning (ECP) and Boundary Value Analysis (BVA) are black-box testing techniques used to design effective test cases. In an online shopping system, these techniques help validate inputs such as product quantity, price, discount, and order amount. Incorrectly grouping valid and invalid inputs into the same equivalence class and ignoring boundary values can allow important defects to remain undetected.

Errors in the Existing Test Design

The tester has incorrectly combined valid and invalid input ranges within the same equivalence class. An equivalence class should contain inputs expected to produce similar system behavior. If valid and invalid values are grouped together, the tester cannot reliably predict the expected result for the entire class.

For example, if the permitted product quantity is from 1 to 100, quantities below 1, quantities from 1 to 100, and quantities above 100 should not be treated as one class.

The second error is the omission of boundary testing. Defects frequently occur at the minimum and maximum permitted values because of incorrect comparison operators, off-by-one errors, or incomplete validation rules.

Root Cause Analysis

The mistakes may have occurred because the tester misunderstood the purpose of equivalence classes or selected test cases without carefully examining the business requirements. Another possible cause is an excessive focus on ordinary inputs instead of values at the limits.

Without boundary testing, the application may incorrectly accept invalid values or reject valid values at the edges of a permitted range.

Corrected Equivalence Classes

Assume the product quantity must be an integer between 1 and 100, inclusive.

The corrected equivalence classes are:

1. Quantity less than 1: invalid class.

2. Quantity from 1 to 100: valid class.
3. Quantity greater than 100: invalid class.
4. Blank or non-numeric quantity: invalid class.
Each class should be tested using a representative input.
Corrected Boundary Value Test Cases
Test the following quantities:
- 0: invalid, because it is below the minimum.
- 1: valid, because it is the minimum permitted quantity.
- 2: valid, because it is immediately above the minimum.
- 99: valid, because it is immediately below the maximum.
- 100: valid, because it is the maximum permitted quantity.
- 101: invalid, because it exceeds the maximum.
The limits are illustrative and should be replaced if the actual shopping system specifies different constraints.
Improved Testing Strategy
The tester should first identify the permitted input ranges from the requirements. Next, divide the inputs into valid and invalid equivalence classes. Then apply BVA to the minimum and maximum values.
Additional tests should verify product availability, stock limits, discount calculations, order totals, and error messages. Regression testing should be performed after any validation defects are corrected.
Conclusion
The existing test design is inadequate because it mixes valid and invalid inputs and ignores important boundaries. Correctly applying ECP and BVA improves test coverage, reduces redundant test cases, and increases the likelihood of detecting input-validation defects before the application is released.

QUESTION 2: Error Analysis of Random Testing in a Banking Application
Introduction
Random testing selects input values without following a predefined systematic test design. Although it can discover unexpected defects, relying entirely on random testing is inadequate for a banking application where transaction limits and balance conditions must be tested precisely.
Errors in the Existing Testing Strategy
The primary error is assuming that random input selection automatically provides high test coverage. A large number of randomly generated test cases does not guarantee that every important boundary or business rule has been exercised.
The testing strategy also fails to distinguish valid and invalid transaction ranges systematically. Consequently, critical defects near minimum and maximum transaction limits may remain undetected.
Root Cause Analysis
Random testing may repeatedly generate ordinary transaction amounts while rarely selecting the exact minimum, maximum, or immediately adjacent values.
It also lacks a direct connection between test cases and specified business requirements unless the random tests are constrained and recorded appropriately.
For example, randomly testing amounts between ₹500 and ₹5,000 may fail to detect a defect that occurs only at a maximum limit of ₹10,000.
Corrective Black-Box Testing Techniques
1. Equivalence Class Partitioning
Divide transaction amounts into meaningful classes:
- Negative amounts: invalid.
- Zero amount: invalid.

- Amount below the minimum: invalid.
- Amount within the permitted range: valid.
- Amount above the maximum: invalid.
- Amount exceeding the available balance: invalid.
- Blank or non-numeric input: invalid.

Select representative values from each class.

2. Boundary Value Analysis

If the permitted transaction range is L to U, test:

- $L - 1$, L, and $L + 1$.
- $U - 1$, U, and $U + 1$.

Use the smallest supported currency unit when selecting adjacent values.

Also test amounts equal to and immediately above or below the available balance.

3. Decision Table Testing

Test combinations involving valid account status, sufficient balance, permitted transaction amount, and valid authorization.

For example, a transaction should be rejected if the account is blocked, even if the balance is sufficient. The expected behavior for every combination must follow the banking requirements.

Improved Testing Strategy

Random testing should remain a supplementary technique for discovering unexpected inputs. The main test suite should use ECP, BVA, decision tables, and risk-based testing.

Every transaction test should verify the response, final account balance, transaction history, and duplicate-processing behavior. Failed transactions must not produce unintended balance changes.

Conclusion

Random testing alone cannot guarantee adequate coverage of critical transaction limits. Combining structured black-box testing techniques with supplementary random testing improves defect detection, test repeatability, and banking transaction reliability.

QUESTION 3: Error Analysis of an Incomplete Control Flow Graph and Path Testing Strategy

Introduction

White-box testing examines a program's internal structure, including decisions, loops, and execution paths. An incomplete Control Flow Graph (CFG) can lead to inadequate path coverage because important branches and loop conditions are omitted from the testing model.

Errors in the Existing Control Flow Graph

The developer has failed to represent all decision nodes and loops. This can result in missing True and False branches, incorrect identification of execution paths, and failure to test loop termination conditions.

If a loop is omitted from the CFG, the testing team may test only a single execution instead of examining repeated iterations and exit behavior.

Root Cause Analysis

The root causes may include incomplete examination of the source code, misunderstanding of control-flow representation, or constructing the CFG using only the normal execution trace.

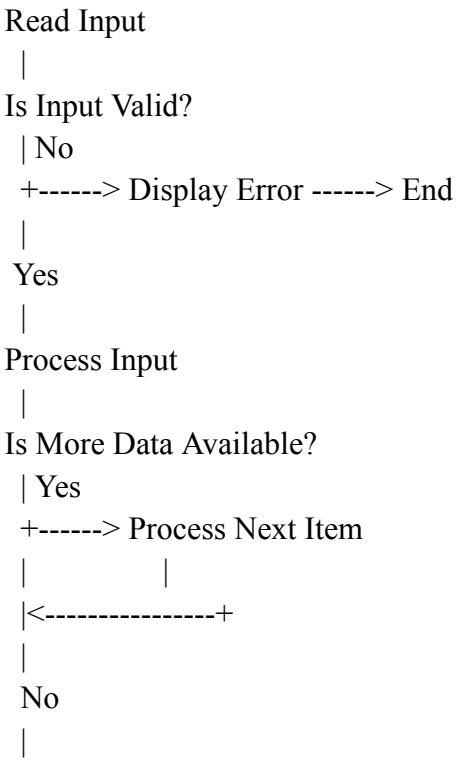
A normal execution trace does not necessarily reveal all possible branches because error conditions and alternative decisions may never occur during that run.

Corrected Control Flow Representation

Consider the following illustrative program logic:

Start

|



This conceptual CFG contains an input-validation decision, an error path, and a loop for processing multiple data items. The actual CFG must represent every decision and loop in the program being tested.

Identification of Independent Paths

The following paths should be tested:

Path 1: Valid input with no repeated processing. The input is valid, processing completes, and the result is displayed.

Path 2: Invalid input. The input-validation decision follows the error branch and terminates safely.

Path 3: Valid input with repeated processing. The loop executes multiple times before reaching its exit condition.

Path 4: Loop termination. The loop reaches its exit condition and the program displays the final result.

The exact number of independent paths depends on the actual source code and CFG.

Cyclomatic Complexity

Cyclomatic complexity estimates the number of linearly independent paths:

$$V(G)=E-N+2P$$

Where E is the number of edges, N is the number of nodes, and P is the number of connected components.

For a single connected graph:

$$V(G)=E-N+2$$

This value guides the number of independent basis paths to test.

Corrected Testing Strategy

The developer should reconstruct the CFG from the source code, identify all decisions and loop edges, and design test cases for each independent path.

Testing should include valid inputs, invalid inputs, both decision outcomes, zero loop iterations where

possible, one iteration, multiple iterations, and loop termination. Error-handling paths should also be tested. Complete path coverage may be impractical for unrestricted loops, so basis-path testing and loop-boundary testing should be used where appropriate.

Conclusion

An incomplete CFG leads to missing branches and inadequate path testing. Correcting the graph and systematically testing independent paths improves branch coverage, reveals hidden control-flow defects, and increases software reliability.

QUESTION 4: Why 100% Statement Coverage Is Insufficient

Introduction

Statement coverage measures whether every executable statement has been executed at least once. Although achieving 100% statement coverage is useful, it does not guarantee that every decision outcome has been tested. Therefore, logical errors may remain undetected even when the statement coverage report shows 100%.

Errors in the Existing Testing Approach

The main error is treating statement coverage as sufficient evidence of software correctness.

A test suite may execute every statement through the normal execution path without executing the alternative branches of conditional statements.

For example:

```
IF balance >= amount
   Approve transaction
ELSE
   Reject transaction
```

If the test suite includes only transactions with sufficient balance, the approval statement executes, but the rejection branch remains untested.

Root Cause Analysis

The root cause is the failure to distinguish statement coverage from branch coverage and path coverage.

Statement coverage focuses on executable statements, whereas branch coverage focuses on decision outcomes. A program may achieve 100% statement coverage without achieving 100% branch coverage.

This is especially important when the application contains nested conditions, financial calculations, validation rules, or error-handling logic.

Improved White-Box Testing Strategies

1. Branch Coverage

Branch coverage requires each decision outcome to execute at least once.

For the balance-validation example, test one transaction where the balance is sufficient and another where the balance is insufficient.

The expected results are transaction approval and transaction rejection, respectively.

2. Path Testing

Path testing examines sequences of decisions through the program.

For a transaction module, test valid authentication followed by a successful transaction, valid authentication followed by insufficient balance, and invalid authentication followed by transaction rejection.

3. Basis-Path Testing

Construct a CFG and calculate cyclomatic complexity to identify independent execution paths. Design test cases to execute each selected basis path.

4. Boundary Value Analysis

Test transaction amounts at the minimum and maximum limits and values immediately below and above

them.

5. Regression Testing

After fixing identified defects, rerun the relevant test cases to ensure that previous functionality continues to work correctly.

Justification of the Improved Approach

Branch coverage identifies untested decision outcomes, while path testing examines combinations of decisions. Boundary testing targets errors near input limits. Together, these techniques provide stronger evidence than statement coverage alone.

However, even 100% branch coverage does not guarantee the absence of all defects. Requirements-based, security, integration, and risk-based testing remain necessary.

Conclusion

The testing team's 100% statement coverage result is insufficient because logical errors can remain in untested branches. Adding branch coverage, basis-path testing, BVA, and regression testing improves defect detection and increases confidence in the module's correctness.

QUESTION 5: Correcting Equivalence Classes for an Age-Validation Field

Introduction

An age-validation field accepts ages from 0 to 100, inclusive. The tester has defined only two equivalence classes: negative numbers and positive numbers. This partitioning is incorrect because it does not distinguish valid ages from positive values outside the permitted range.

Errors in the Existing Partitioning Strategy

The main error is grouping all positive numbers into one class. Although ages from 1 to 100 are valid, ages greater than 100 are invalid.

The existing classes also fail to identify zero explicitly, even though zero is valid under the stated range of 0 to 100.

Additionally, the strategy does not account for non-numeric inputs, blank values, or boundary values.

Root Cause Analysis

The tester has considered only whether a number is positive or negative instead of using the actual input constraints.

This misunderstanding can cause invalid values such as 150 to be accepted and valid boundary values such as 0 or 100 to be mishandled.

Corrected Equivalence Classes

The correct classes are:

1. Age less than 0: invalid.
2. Age from 0 to 100, inclusive: valid.
3. Age greater than 100: invalid.
4. Blank input: invalid.
5. Non-numeric input: invalid.

Representative values should be selected from each class.

Application of Boundary Value Analysis

The following boundary values should be tested:

- −1: invalid.
- 0: valid minimum age.
- 1: valid age immediately above the minimum.
- 99: valid age immediately below the maximum.
- 100: valid maximum age.

- 101: invalid.

The system should also reject blank and non-numeric values with appropriate validation messages.

Justification

The corrected equivalence classes distinguish valid and invalid values according to the specified range. BVA checks the exact points where input validity changes.

Together, these methods detect off-by-one errors, incorrect comparison operators, and invalid input acceptance without requiring tests for every possible age.

Conclusion

The original partitioning is incomplete because it groups valid and invalid positive values together and ignores the exact boundaries. Corrected ECP and BVA provide more reliable age validation and improve the effectiveness of the test suite.

QUESTION 6: Condition Coverage for a Complex Boolean Expression

Important note: The uploaded question paper shows only the beginning of this question: A developer claims full Condition Coverage for a complex if(A && B. The remaining question is not visible. The following answer is based on the assumption that the expression is if (A && B) and that the task asks whether condition coverage is adequate.

Introduction

Condition coverage is a white-box testing technique that checks whether each individual Boolean condition evaluates to both True and False at least once. For a compound expression such as if (A && B), testing each condition is important, but the relationship between condition coverage and decision coverage must also be understood.

Analysis of the Testing Claim

For the expression:

if (A && B)

the overall decision is True only when both A and B are True. Otherwise, the decision is False.

A developer may claim full condition coverage after making each condition evaluate to True and False. However, the claim should be verified against the actual test cases and decision outcomes.

Example of Condition Coverage

Consider the following tests:

- Test 1: $A = True, B = True$. The decision is True.
- Test 2: $A = False, B = True$. The decision is False.
- Test 3: $A = True, B = False$. The decision is False.

Across these tests, both A and B evaluate to True and False. The overall decision also takes both outcomes.

Root Cause Analysis

The main testing risk is assuming that condition coverage alone proves that all logical behavior has been tested. In larger expressions containing multiple conditions, condition coverage may not demonstrate that every decision outcome occurs or that each condition independently affects the overall decision.

Also, in programming languages that use short-circuit evaluation, the second condition may not be evaluated when the first condition is False. Test design and coverage measurement should account for the language's evaluation rules.

Improved Testing Strategy

The developer should document each atomic condition, construct test cases for its True and False outcomes, and verify the overall decision result.

For more complex safety-critical decisions, Modified Condition/Decision Coverage (MC/DC) can be used to demonstrate that each condition independently affects the decision outcome.

Conclusion

Condition coverage is useful for identifying untested Boolean conditions, but it should not automatically be treated as proof of complete logical testing. The developer should verify condition outcomes, decision outcomes, and, where necessary, independent condition effects using MC/DC.

QUESTION 7: Error Analysis of an Incomplete Decision Table for a Discount System

Introduction

A discount system uses the conditions "Premium Member" and "Coupon Applied" to calculate prices. Missing combinations of these conditions can cause incorrect discount calculations. Decision Table Testing is suitable because it maps condition combinations to their expected actions.

Errors in the Existing Decision Table

The QA team has omitted some combinations of membership status and coupon usage. This means that the test suite does not represent all possible combinations of the specified Boolean conditions.

For two independent Boolean conditions, there are four combinations:

1. Premium member and coupon applied.
2. Premium member and no coupon applied.
3. Non-premium member and coupon applied.
4. Non-premium member and no coupon applied.

If any combination is omitted, the corresponding business rule may remain untested.

Root Cause Analysis

The incomplete table may have been created by testing only common scenarios or by assuming that membership and coupon eligibility are independent when the discount rules actually interact.

For example, a coupon might be available only to premium members, or premium members might receive a different discount when a coupon is applied. These rules must be explicitly represented.

Reconstructed Decision Table

The four required condition combinations and their actions are described below.

Rule 1: Premium member, coupon applied.

Expected action: Calculate the premium-member discount and apply the coupon according to the specified stacking rules.

Rule 2: Premium member, no coupon applied.

Expected action: Apply the premium-member discount only.

Rule 3: Non-premium member, coupon applied.

Expected action: Apply the coupon if eligible, or reject it if the rules restrict coupons to premium members.

Rule 4: Non-premium member, no coupon applied.

Expected action: Calculate the standard price without membership or coupon discounts.

The exact discount amounts and coupon-eligibility rules must be obtained from the business requirements; they are not specified in the question.

Improved Testing Strategy

For every rule, verify the original price, applicable discount, final price, and any rejection message.

Additional tests should include invalid coupons, expired coupons, zero-priced products, maximum permitted discounts, and combinations where discounts cannot be stacked.

The tester should compare the results with independently calculated expected values and rerun the tests after changes to the discount logic.

Conclusion

The original decision table is incomplete because it omits combinations of membership and coupon conditions. Testing all four combinations and verifying each action improves business-rule coverage and

reduces the risk of incorrect prices in production.

QUESTION 8: Error Analysis of Loop Testing

Introduction

Loop testing examines how a program behaves when loops execute zero, one, multiple, or maximum permitted iterations. In this scenario, the tester checks only the first iteration ($n = 1$) and the maximum iteration ($n = max$), missing defects at $n = 0$ and $n = max + 1$.

Errors in the Existing Strategy

The tester has omitted the zero-iteration case and the value immediately above the maximum permitted iteration count.

Testing only the first and maximum iterations does not verify whether the loop handles empty input correctly or rejects excessive iteration counts.

The strategy may also overlook errors involving repeated iterations, termination conditions, loop counters, and interactions between nested loops.

Root Cause Analysis

The root cause is an incomplete understanding of loop boundaries. Defects often occur when a loop condition uses an incorrect comparison operator, when the loop counter is initialized incorrectly, or when the counter is not updated correctly.

For example, a loop intended to execute while $i < n$ may accidentally use $i \leq n$, causing one extra iteration.

Corrected Testing Strategy for Simple Loops

For a simple loop, test:

1. Zero iterations, where permitted.
2. One iteration.
3. Two iterations.
4. A typical number of iterations.
5. The maximum permitted number of iterations.
6. One iteration above the maximum, which should be rejected or handled according to the requirements.

Also test invalid loop bounds and conditions that may cause non-termination.

Testing Strategy for Nested Loops

For nested loops, begin by testing the inner loop independently. Then test combinations where the outer loop executes zero, one, and multiple times while the inner loop executes zero, one, and multiple times where permitted.

Finally, test the combined nested structure using representative and boundary-sized inputs.

Monitor execution time and verify that each loop terminates correctly. Use timeouts for potentially non-terminating paths.

Improved Path Integrity

Loop tests should verify the correct entry condition, iteration count, counter update, exit condition, and final result. For nested loops, tests should also verify that the inner loop resets and executes correctly for each outer iteration.

Basis-path testing can supplement loop-boundary testing when the program has multiple independent branches.

Conclusion

Testing only $n = 1$ and $n = max$ leaves important loop boundaries untested. Adding zero-iteration, repeated-iteration, maximum-plus-one, and nested-loop scenarios improves loop correctness and helps detect off-by-one errors, incorrect termination, and path-integrity defects.

QUESTION 9: Error Analysis of Error Guessing in a Safety-Critical Braking System

Introduction

A braking system is safety-critical because software defects can directly affect vehicle control and human safety. Error Guessing relies on a tester's experience to anticipate likely failures. However, using it as the primary technique while ignoring structured black-box testing creates significant gaps in test adequacy.

Errors in the Existing Strategy

The main error is relying on an ad-hoc technique without systematic coverage of specified input ranges and system states.

Error Guessing may identify known failure scenarios, but it does not guarantee that all important boundary values or state transitions are tested. It also makes it difficult to demonstrate systematic requirements coverage.

Root Cause Analysis

The strategy may be based on past experience rather than a documented test model. As a result, unusual operating conditions, exact thresholds, and transitions between braking states may remain untested.

In a braking system, this could include incorrect behavior when sensor readings approach a threshold or when the system changes between normal braking and emergency braking.

Application of Boundary Value Analysis

BVA should be applied to safety-relevant numerical inputs, such as permitted sensor ranges, braking thresholds, and response-time limits defined by the requirements.

For a threshold T, test values immediately below T, exactly at T, and immediately above T. Repeat the process for applicable minimum and maximum values.

The expected response for each boundary must be determined by the system's safety specification.

Application of State Transition Testing

Model the braking system's relevant states, such as:

- Normal operation.
- Braking requested.
- Braking active.
- Emergency braking.
- Sensor fault.
- Fault recovery.

The actual states and transitions must follow the system design.

Test valid transitions, such as normal operation to braking active, and safety-related transitions, such as sensor fault to the specified safe state.

Also test invalid or unexpected sequences, delayed sensor updates, repeated braking requests, and recovery after a fault.

Improved Testing Strategy

Combine BVA and State Transition Testing with requirements-based tests, fault injection, and Error Guessing as a supplementary technique.

Every test should verify the expected state, control output, timing behavior, and fault response. Testing should be performed in a controlled simulation or hardware-in-the-loop environment before any authorized real-world validation.

Safety requirements, traceability, independent review, and applicable industry standards must guide the test process. No single testing technique can establish that a safety-critical system is completely safe.

Conclusion

Error Guessing alone is inadequate for a safety-critical braking system because it cannot guarantee

systematic coverage of boundaries and state transitions. Combining BVA and State Transition Testing with risk-based and requirements-based validation improves test adequacy and provides stronger evidence of safety-related behavior.

QUESTION 10: Error Analysis of a Control Flow Graph with Missing Else Branches

Introduction

A Control Flow Graph (CFG) represents the execution flow of a program, including statements, decisions, branches, loops, and termination points. In this scenario, the tester omits the else branches of nested if statements, resulting in 100% statement coverage but no branch coverage on error paths.

Errors in the CFG Construction

The primary error is the omission of alternative decision outcomes. Every conditional statement must represent both the True and False outcomes, even when one branch contains only an error message or an early return.

The second error is constructing the graph around the normal execution path rather than all possible execution paths.

As a result, the CFG does not accurately represent the program's error handling.

Root Cause Analysis

The likely root causes include incomplete examination of the source code, misunderstanding the role of decision nodes, and assuming that executing every statement guarantees branch coverage.

If the normal path executes all major processing statements, statement coverage may reach 100% even though alternative branches remain untested.

For nested conditions, omitted else branches may contain important validation failures, authorization rejections, or exception-handling operations.

Corrected Control Flow Graph

Consider the following illustrative nested logic:

Start
 |
Validate Input
 |
Input Valid?
 | No
 +------> Display Input Error ------> End
 |
 Yes
 |
Credentials Valid?
 | No
 +------> Display Authentication Error ------> End
 |
 Yes
 |
Transaction Allowed?
 | No
 +------> Reject Transaction ------> End
 |
 Yes

|
Execute Transaction
|
Record Result
|
End

This conceptual CFG includes alternative paths for invalid input, failed authentication, and rejected transactions. The actual graph must include all decisions and branches present in the source code.

Identification of Independent Paths

The following paths should be tested:

Path 1: Successful execution. Input is valid, credentials are valid, and the transaction is permitted.

Path 2: Invalid input. The first decision follows the error branch and terminates safely.

Path 3: Invalid credentials. Input validation succeeds, but authentication fails.

Path 4: Transaction rejected. Input and authentication are valid, but the transaction is not permitted.

Each path should verify the expected output, system state, error handling, and absence of unintended side effects.

Cyclomatic Complexity and Coverage

Calculate cyclomatic complexity using:

$$
V(G)=E-N+2P
$$

For a single connected graph:

$$
V(G)=E-N+2
$$

Use this measure to help identify the number of independent basis paths. Then design tests that execute both outcomes of every decision.

Branch coverage should be measured against the actual source code and CFG. The reported 0% branch coverage on error paths indicates that the alternative outcomes have not been adequately exercised.

Improved Testing Strategy

Reconstruct the CFG from the source code and include every if, else, nested condition, loop, and early return. Create test cases for all decision outcomes, including error paths.

After corrections, measure branch coverage, perform basis-path testing, and execute regression tests. Verify that invalid inputs and failed authorization do not execute transaction-processing statements.

Conclusion

Omitting else branches creates an incomplete CFG and leaves important error-handling paths untested. Correcting the graph and testing every decision outcome improves branch adequacy, identifies hidden logical defects, and provides stronger evidence of program reliability.